PRACT 1 UBER

October 31, 2026

[6]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

[7]:
```
df = pd.read_csv(r"E:\LP3_LAB Manual\ML PRAC\PRAC1\uber.csv")
```

[8]:
```
df.head()
```

[8]:

	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count
0	24238194	2015-05-07 19:52:06	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.738354	-73.999512	40.723217	1
1	27835199	2009-07-17 20:04:56	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.728225	-73.994710	40.750325	1
2	44984355	2009-08-24 21:45:00	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.740770	-73.962565	40.772647	1
3	25894730	2009-06-26 08:22:21	5.3	2009-06-26 08:22:21 UTC	-73.976124	40.790844	-73.965316	40.803349	3
4	17610152	2014-08-28 17:47:00	16.0	2014-08-28 17:47:00 UTC	-73.925023	40.744085	-73.973082	40.761247	5

[9]:
```
df=df.drop(["Unnamed: 0"],axis=1)
df.head()
```

[9]:

	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count
0	2015-05-07 19:52:06	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.738354	-73.999512	40.723217	1
1	2009-07-17 20:04:56	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.728225	-73.994710	40.750325	1
2	2009-08-24 21:45:00	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.740770	-73.962565	40.772647	1
3	2009-06-26 08:22:21	5.3	2009-06-26 08:22:21 UTC	-73.976124	40.790844	-73.965316	40.803349	3

```
4  2014-08-28 17:47:00         16.0  2014-08-28 17:47:00 UTC

   pickup_longitude  pickup_latitude  dropoff_longitude  dropoff_latitude  \
0        -73.999817        40.738354         -73.999512         40.723217
1        -73.994355        40.728225         -73.994710         40.750325
2        -74.005043        40.740770         -73.962565         40.772647
3        -73.976124        40.790844         -73.965316         40.803349
4        -73.925023        40.744085         -73.973082         40.761247

   passenger_count
0                1
1                1
2                1
3                3
4                5
```

[10]:
```
df=df.drop(["key"],axis=1)
df=df.drop(["pickup_datetime"],axis=1)
df.head()
```

[10]:
```
   fare_amount  pickup_longitude  pickup_latitude  dropoff_longitude  \
0          7.5        -73.999817        40.738354         -73.999512
1          7.7        -73.994355        40.728225         -73.994710
2         12.9        -74.005043        40.740770         -73.962565
3          5.3        -73.976124        40.790844         -73.965316
4         16.0        -73.925023        40.744085         -73.973082

   dropoff_latitude  passenger_count
0         40.723217                1
1         40.750325                1
2         40.772647                1
3         40.803349                3
4         40.761247                5
```

[11]:
```
df.isnull().sum()
```

[11]:
```
fare_amount          0
pickup_longitude     0
pickup_latitude      0
dropoff_longitude    1
dropoff_latitude     1
passenger_count      0
dtype: int64
```

[12]:
```
df=df.dropna()
```

[13]:
```
df.isnull().sum()
```

[13]:
```
fare_amount          0
pickup_longitude     0
pickup_latitude      0
dropoff_longitude    0
dropoff_latitude     0
passenger_count      0
dtype: int64
```

[14]:
```
# Identifying outliers
def detect_outliers(df,feature):
    q1=df[feature].quantile(0.25)
    q3=df[feature].quantile(0.75)
    iqr=q3-q1
    lower_limit=q1-1.5*iqr
    upper_limit=q3+1.5*iqr
    return df[(df[feature]<lower_limit)|(df[feature]>upper_limit)]
```

[15]:
```
detect_outliers(df,"fare_amount")
```

[15]:

	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count
6	24.50	-73.961447	40.693965	-73.871195	40.774297	5
30	25.70	-73.944815	40.834367	-73.989332	40.721920	4
34	39.50	-73.788080	40.642187	-73.865042	40.725997	4
39	29.00	-73.992600	40.753172	-73.908508	40.816192	1
48	56.80	-73.993498	40.764686	-73.993498	40.764686	1
...	...	...	...	...	...	...
199976	49.70	-73.978225	40.783318	-73.700963	40.705852	1
199977	43.50	-73.996671	40.737483	-73.867758	40.897563	1
199982	57.33	-73.969204	40.754771	-73.790351	40.643802	1
199985	24.00	-74.005089	40.737301	-73.945290	40.774162	5
199997	30.90	-73.986017	40.756487	-73.858957	40.692588	2

[17166 rows x 6 columns]

[16]:
```
#Remove Outliers
def remove_outliers(df,feature):
    q1=df[feature].quantile(0.25)
    q3=df[feature].quantile(0.75)
    iqr=q3-q1
    lower_limit=q1-1.5*iqr
    upper_limit=q3+1.5*iqr
    return df[(df[feature]>lower_limit)|(df[feature]<upper_limit)]
```

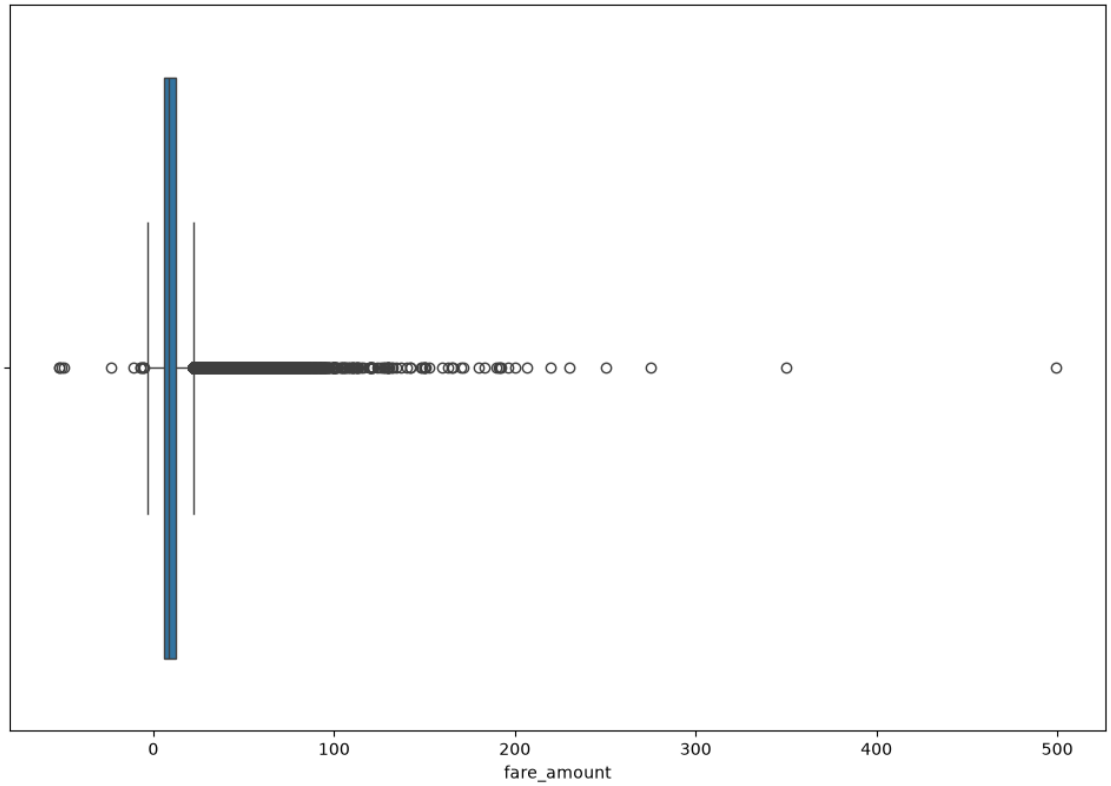
[17]:
```
remove_outliers(df,"fare_amount")
```

[17]:

	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count
0	7.5	-73.999817	40.738354	-73.999512	40.723217	1
1	7.7	-73.994355	40.728225	-73.994710	40.750325	1
2	12.9	-74.005043	40.740770	-73.962565	40.772647	1
3	5.3	-73.976124	40.790844	-73.965316	40.803349	3
4	16.0	-73.925023	40.744085	-73.973082	40.761247	5
...	...	...	...	...	...	...
199995	3.0	-73.987042	40.739367	-73.986525	40.740297	1
199996	7.5	-73.984722	40.736837	-74.006672	40.739620	1
199997	30.9	-73.986017	40.756487	-73.858957	40.692588	2
199998	14.5	-73.997124	40.725452	-73.983215	40.695416	1
199999	14.1	-73.984395	40.720077	-73.985508	40.768793	1

[199999 rows x 6 columns]

[21]:
```
plt.figure(figsize=(12,8))
sns.boxplot(x="fare_amount",data=df)
plt.show()
```

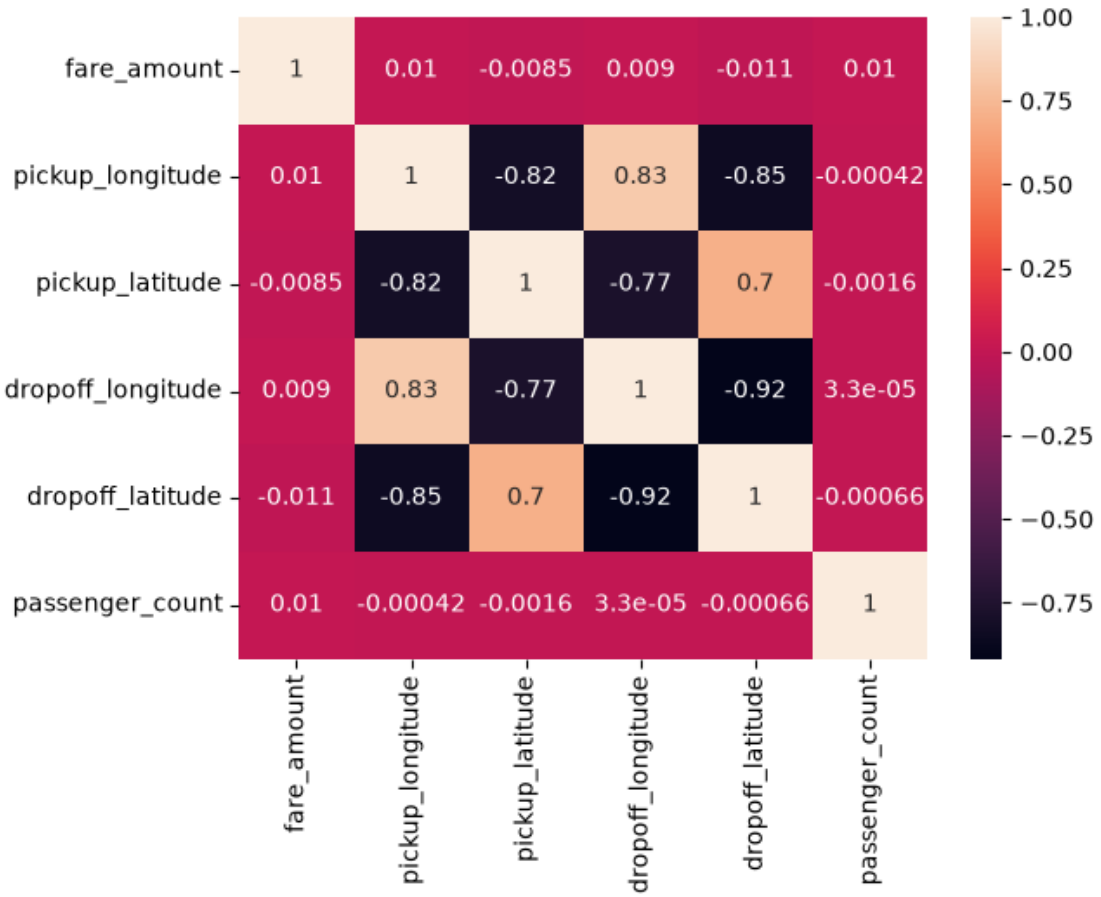


[19]:
```
#Check for correlation
```

[22]:
```
corr=df.corr()
```

[23]:
```
sns.heatmap(corr,annot=True)
```

[23]: <Axes: >



[24]:
```
# importing packages from sklearn
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

[25]:
```
# Remove invalid fares
df = df[df["fare_amount"] > 0]
```

[26]:
```
# Input and target
X = df.drop("fare_amount", axis=1)
y = df["fare_amount"]
```

[27]:
```
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

[28]:
```
# Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
lr_pred = lr.predict(X_test)

rmse=np.sqrt(mean_squared_error(y_test,lr_pred))
r2_score=r2_score(y_test,lr_pred)
print("Linear Regression")
print("RMSE:", rmse)
print("R2 Score:", r2_score)
```

```
Linear Regression
RMSE: 9.64284319045685
R2 Score: 3.586364757424132e-05
```

[29]:
```
from sklearn.metrics import r2_score
from sklearn.ensemble import RandomForestRegressor
# Random Forest Regression
rf = RandomForestRegressor(n_estimators=50, random_state=42)

rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

r = np.sqrt(mean_squared_error(y_test, rf_pred))
r2 = r2_score(y_test, rf_pred)

print("Random Forest Regression")
print("RMSE:", r)
print("R2 Score:", r2)
```

```
Random Forest Regression
RMSE: 4.451778033254697
R2 Score: 0.7868716514256658
```

[30]:
```
# Predict fare for a new ride
new_ride = pd.DataFrame([{
    "pickup_longitude": -73.99,
    "pickup_latitude": 40.73,
    "dropoff_longitude": -73.98,
    "dropoff_latitude": 40.75,
    "passenger_count": 2
}])

print("\nLinear Regression Predicted Fare:",
      lr.predict(new_ride)[0])

print("Random Forest Predicted Fare:",
      rf.predict(new_ride)[0])
```

```
Linear Regression Predicted Fare: 11.366354231148778
Random Forest Predicted Fare: 8.098
```

[ ]: